

CostVision and CAPEE

The business case for automating cost input, and then costing in bulk

1

Feed CAPEE automatically, and prove it

CAPEE keeps doing the costing. CostVision reads the numbers off the 3D model and the drawing and fills them in. We prove it on real JLR parts first.

2

Cost in bulk, run by an agent

A whole basket of parts costed in one run, unattended, with a person asked only where the geometry genuinely cannot decide.

THE DECISION

What we are asking for

Approve a proof of concept: Option 1, inside CAPEE, on our own parts

Let CostVision fill CAPEE's cost input automatically for one part type, on 30 to 50 parts we have already bought. At the end we will know how often the measurement is right, how often an engineer has to correct it, and what it saves. Option 2 follows once that is known.

	Detail
What we want to do	Fill CAPEE's cost input from the 3D model and the drawing, for one part type
What we need from JLR	30 to 50 parts with the price we paid, our own rate data, three answers from IT, and a team for the trial
What it costs to try	Our own engineering time. No licence, no new tool for the business to learn, CAPEE unchanged
What we will know at the end	A measured accuracy figure against real JLR prices, and a measured time saving per part
What we are NOT asking for	A decision on Option 2 today. It needs the accuracy answer first, and some work that has not been done

The one thing to know before you decide

CostVision has never been checked against a price JLR has actually paid. Not one part. That is exactly what the proof of concept is for, and until it is done I cannot tell you how accurate the tool is.

THE PROBLEM

What costing a part looks like today

TODAY, BY HAND

The engineer opens the part in CAD, reads the drawing, and types the numbers into CAPEE.

Boxes to fill, depending on the part type:

machined part 12 · casting 30 · moulding 39 · pressing 69

Part weight, starting material weight, wall thickness, hole and pocket counts, machine times, surface area, tolerances, finish, heat treatment.

Two engineers costing the same part will not type the same numbers. So one part gets two answers.

WITH COSTVISION

The engineer hands over the 3D model and the drawing.

Measured off the model:

volume, size, surface area, wall thickness, holes, pockets, bosses, gear teeth, bend count.

Read off the drawing:

tolerances, surface finish, heat treatment, coating, masked features.

The tool works out 182 values and puts 139 of them straight into a named box. The same part gives the

Measuring and reading are two different jobs

The 3D model is measured by software, in the way a CMM measures a part. That is not AI. The AI only reads the words written on the drawing, and everything it reads is checked against the measured model before it can move a cost. Slide 8 covers those checks. On the deterministic setting the AI is switched off completely and the part still costs.

CURRENT STATUS

Where CostVision is today

Read off the software this month. This is what the tool does today.

WORKING NOW

19 part types cost through the engine, plus an assembly roll-up and a separate software model
The costing gives the same answer every run
Measures 3D models with no internet connection
Reads drawings for tolerances, finish, heat treatment
Refuses geometry it cannot honestly measure
Asks before it scales an inch model
Same safety checks on every route, printed in the report
328 materials, 178 machines, 42 labour rates, 20 regions
2,079 automatic tests across 156 files, all passing

NOT THERE YET

Never compared against a price JLR has paid
No history kept when someone changes a rate
Finished costings are not saved, so an old one cannot be reproduced
No JLR sign-on; it has its own logins today
Gear costs in the screens but the automated route cannot reach it
The automated route cannot yet be told a region or an annual volume
Still under active development

What changed since the last time I showed you this

Six rounds of work in the last few weeks. The tool now refuses bad geometry instead of costing it, asks about units, runs the same checks on every route and prints them in the report, measures a file once instead of four times, and widens its own confidence range when a number was typed rather than measured.

SEQUENCE

The plan: prove it small, then scale it



OPTION 1 · feed CAPEE automatically

CAPEE stays as it is and still does the costing.
Instead of typing between twelve and seventy boxes,
the engineer hands over the 3D model and the drawing
and the boxes fill in.

**Moves about 5,700 lines of measuring and reading
code. plus the safety checks that go with it.**

Why this order

Option 1 is the cheaper thing to try and it produces the accuracy number that Option 2 needs. Costing five hundred parts is only worth doing once we know the tool gets one part right.

OPTION 2 · cost in bulk, run by an agent

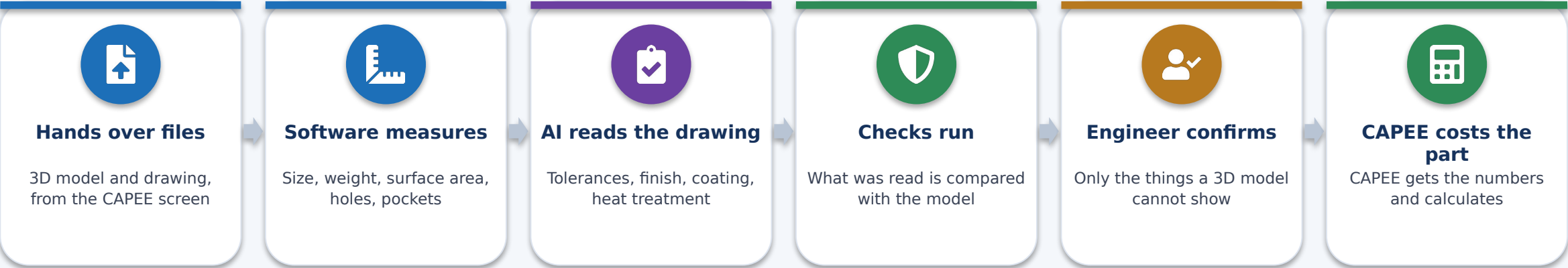
A list of parts goes in. Every one is measured, costed
and reported without anyone sitting there.
An engineer is asked only where the geometry
genuinely cannot decide.

**Same engine, same rates, same checks. The new work
is the run itself and keeping a record of it.**

OPTION 1

Option 1: what happens to one part

What an engineer does each time they cost a part



OPTION 1 · HANDOVER

Option 1: what we hand over

Sizes counted straight out of the software this month. Both languages are in common use and JLR IT will have people who know them.

Part	Language	Size	What it does
Measuring engine	Python	2,462 lines	Opens the 3D model and measures it. Makes no internet connection.
Reading the drawing	TypeScript	830 lines	Sends the drawing to the AI and sorts out what comes back.
Safety checks	TypeScript	536 lines	The four checks on the next slide, in three files.
Questions per part type	TypeScript	8,509 lines	Works out 182 values and decides what the engineer gets asked.
Design-for-cost findings	TypeScript	1,992 lines	Flags features that are expensive to make and says why.
Settings list	Plain list	one file	Every fixed number pulled together in one place. Slide 9.

About the two languages

Python is the normal choice for engineering and measurement work. TypeScript is widely used for business software. Neither is unusual and neither ties JLR to one supplier.

One thing for IT to confirm

The measuring engine needs a normal Linux server, not the cut-down kind used for small services. It decides where this can go, so it is worth settling early.

OPTION 1 · REQUIRED

Option 1: the checks that have to come with it

The AI does not set a price. It does hand over numbers that feed into one, so a bad reading turns into a bad cost. These four checks are what stop that, and they have to come across with everything else.



The measured model beats the AI

If the AI reads 2 kg off the drawing and the model measures 1.4 kg, we use the measured figure. The geometry wins.



Everything read gets cross-checked

Numbers taken off the drawing get compared against the measured model. Where they disagree, the engineer is told and has to accept it by name before the cost stands.



Machining time has a ceiling

On a casting or forging, machining is capped at what finishing that part could realistically take, so a bad estimate cannot run the cost up.



Unknowns get asked, not guessed

Heat treatment, tolerance class and material grade cannot be read off a solid model, so the engineer is asked. The AI suggestion is labelled as a suggestion and never comes pre-ticked.

Two things we found in our own review, and fixed

A route through the tool had no checks on it and returned a cost of "not a number" while reporting success. And a part exported as surfaces rather than a solid was costed at a plausible but wrong weight. Both now stop the costing instead of producing a figure.

OPTION 1 · SETTINGS

Option 1: the fixed numbers inside the software

You asked what the hardcoded values actually are. These are engineering judgements written into the code. Six real ones, read off the software this month:

What it controls	Value	Why
Smallest face counted as machined	400 mm²	Below this we treat it as an edge, not a face to machine
Smallest pocket counted	80 mm²	Stops small recesses getting costed as pockets
Cost of a rejected plated part	4.5x a good one	You strip it and plate it again, so a 3% reject rate is nearer 13% on cost
Zinc price used for plating	USD 3.67 / kg	Market reference, August 2026. Moves monthly and is meant to get updated
Shape allowance, pressing	1.15	A real pressing has more surface than a flat plate: edges, flanges, bends
Shape allowance, casting and forging	1.25 / 1.10	Castings carry ribs and bosses; forgings are simpler on the outside

How they get handed over

Pulled out of the code into one list you can read and change without a developer. That list is a deliverable in its own right and it is a step in the Option 1 sequence.

Where these came from

They are our engineering estimates. They are not measurements from a JLR plant. Treat them as a starting point and expect to replace them as real data comes in.

OPTION 1 · DECISION FOR IT

Option 1: three ways to connect it to CAPEE

We do not know yet how CAPEE is built. All three routes work. The answer decides which one we use and how much effort it takes.

Route	Use when	What it means day to day	Relative effort
A · Put it inside CAPEE	CAPEE is built in JavaScript	Our code goes straight in. Only the measuring engine sits separately.	Least effort
B · Run it alongside	CAPEE is Java, .NET or anything else	It runs as its own service on a JLR server and CAPEE asks it for numbers over a secure link.	Most effort
C · Pass a file	CAPEE is a desktop or spreadsheet tool	It writes a file that CAPEE imports. Simplest to build, but somebody has to move the file.	Least effort

If we had to pick blind

Route B. It works whatever CAPEE turns out to be, and we can update CostVision without touching CAPEE every time.

Three questions for IT

What is CAPEE written in. Can it call another service on the network. Can it run on a normal Linux server.

Effort is shown against the other two routes. None of the three has been sized by JLR.

OPTION 1 · THE TRIAL

Option 1: what the proof of concept proves

One part type, one team, 30 to 50 parts we have already bought. Four numbers come out of it, and none of them exist today.

1

How close is it?

Cost every part with the tool, compare against the price we actually paid, and report the average error. The test is whether the real error lands inside the range the tool itself predicted for each part. A tool that is wrong and says so is usable. One that is wrong and confident is not.

2

How often does the engineer have to step in?

Count the values the engineer changed after the tool filled them, and the parts where the tool stopped and asked. On our own eight sample parts, six costed straight through and two needed an answer. We need that figure on JLR parts, not ours.

3

How much time does it save?

Time the same parts both ways: typed by hand into CAPEE, and filled by the tool with the engineer checking. The difference is the saving, measured rather than claimed.

4

What does it refuse, and was it right to?

Every part the tool turned down, with the reason. If it refuses parts it should have costed, that is a fault we need to see early.

Why it is worth doing even if the answer is disappointing

A measured accuracy figure is worth having either way. Today nobody can state one, so every conversation about this tool stalls in the same place.

OPTION 1 · SEQUENCE

Option 1: order of work

1

Agree the connection

Find out how CAPEE is built, pick one of the three routes, agree exactly which numbers CAPEE expects.

2

Stand up the measuring software

Install it on a JLR server. Measure 20 real JLR parts and check the results against their drawings.

3

Hand over the settings list

Pull every fixed number into one list. JLR review it and sign it off before anything gets costed with it.

4

Move the safety checks across

The four checks and the questions the engineer confirms. In the scope from the start.

5

Connect into CAPEE

Wire it in. Route the drawing reading through a JLR-approved AI service. Cost one real part end to end.

6

Run the trial

One team, one part type, 30 to 50 parts with prices we paid. Produce the four numbers from the last slide.

Steps 1 and 2 can start together

Standing up the measuring software does not depend on knowing how CAPEE is built. Only step 5 does.

Start with one part type

Machined parts or pressings first. Most of our volume and the shapes we understand best. Widen it once it holds up.

OPTION 2

Option 2: costing in bulk, run by an agent

WHAT WE DO TODAY

We cost the parts we have time to cost.

A basket of several hundred parts gets sampled, or estimated by analogy against something similar, or taken from what the supplier last quoted.

Nobody costs all of it from the geometry, because there are not enough engineer-hours to do it.

WHAT OPTION 2 DOES

A list of parts goes in. Every one is measured, costed and reported in one run, unattended.

An engineer is asked only where the geometry genuinely cannot decide, and only once per question.

Same engine, same rates, same safety checks as a single part. Nothing about the maths changes.

What the agent actually does, and what it does not

The agent marshals the run. It picks the part type, answers what it can from the drawing, and escalates what it cannot. Every number in the answer still comes from the same deterministic engine, at the same rates, with the same checks. The agent never sets a price. That rule does not bend for bulk.

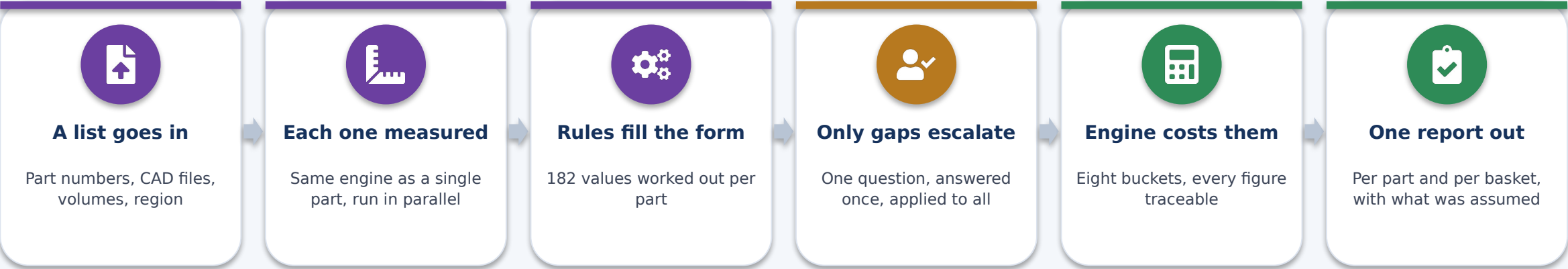
Where this stands today

The engine is already callable on its own, and there is already an agent that can cost a part when asked. What does not exist is the run itself: a list going in, a record coming out, and a way to reproduce it later. Slide 15 sets out exactly what has to be built.

OPTION 2

Option 2: what a bulk run does

What happens to a basket of parts, start to finish



How long a run takes, measured on our own parts this month

What	Measured	Basis
Measure one part, warm	1.2 seconds	Average over eight real parts, 0.6 MB to 20 MB
Work out the 182 values and cost it	5 milliseconds	Rules plus the eight-bucket engine. Not the slow part
100 parts	1 minute	On two measuring workers, unattended
500 parts	5 minutes	On two workers. Two and a half on four
1,000 parts	10 minutes	On two workers. Five on four

The machine time is not the constraint

Getting the part list, the CAD files and the rate data together is the work. The computer finishes in minutes.

OPTION 2 · PREREQUISITES

Option 2: what has to be built first

These are the honest gaps. None of them is large on its own, and none of them is optional.

What is missing	Why it matters	Detail	Type
A history of rate changes	Blocks audit	Today the rate book is one row that gets overwritten. Change a rate and the old one is gone. A bulk run costed at rates nobody can recover is not defensible.	Must have
Saved costings	Blocks audit	Finished costings are not stored anywhere. A five-hundred-part run would produce numbers with nowhere to live and no way to reproduce them.	Must have
Region and volume on the automated route	Blocks correctness	The automated route always uses the UK rate book and never gets told the annual volume. Both already work in the screens; they are simply not passed through.	Must have
Gear on the automated route	Blocks coverage	Gear costs in the screens and has its own rules, but the automated route cannot reach it. A basket with gears in it would silently skip them.	Must have
The run itself	The new work	A list in, a job queue, a report out, and a way to answer one question once and apply it to every part it affects.	New build

The good news underneath all of that

The costing engine itself needs no change. We tested it on its own, outside the application, and it produced the full eight-bucket answer with nothing adjusted. The work above sits around the engine.

OPTION 2 · THE CASE

Option 2: what it changes for the business

	Today	With bulk costing
How many parts get a proper cost	The ones we have hours for	All of them
How a basket gets priced	Sample and scale up	Every part costed from its own geometry
Turnaround on a new basket	Days of engineer time	Minutes of machine time, plus the data gathering
Consistency across a basket	Depends who costed it	Same rules and rates for every part
What we can say to a supplier	This is our estimate	Here is the build-up, per part, with the rates
Where an engineer spends the time	Typing numbers in	Answering the questions only a person can
What it needs from a person	One engineer per part	One answer per question, applied to every part

The honest limit on all of this

Every row assumes the tool is accurate enough to trust unattended, and today nobody can say whether it is. That is precisely what Option 1 measures, which is why it comes first.

OPTION 2 · SEQUENCE

Option 2: order of work

1

Load JLR rates

Materials, machine rates, labour, energy, country factors. This sets the pace and it is ours to do.

2

Keep a history of rate changes

Every change stamped and kept, so a costing can name the rate book it used. Prerequisite for anything audited.

3

Save finished costings

A stored record per part: inputs, rates, answers, result. Without this a bulk run cannot be reproduced.

4

Close the automated-route gaps

Pass region and annual volume through, and register gear. Small work, and a basket is wrong without it.

5

Build the run

A list in, a job queue, one question answered once, a report out per part and per basket.

6

Trial on a real basket

One programme, one commodity family, run beside the way we cost today and compare.

Steps 2 and 3 are the real prerequisites

They are not part of the run, and they are what makes a run mean anything afterwards.

Step 1 can start now

Gathering our own rate data helps both options and does not wait on any decision about Option 2.

COMPARISON

The two options together

	OPTION 1 · feed CAPEE	OPTION 2 · bulk costing
What it gives us	The engineer stops typing	We can cost a whole basket
Which system does the costing	CAPEE, unchanged	CostVision engine, beside CAPEE
Does anyone learn a new tool	No, same CAPEE screens	A reviewer does, for the results
How much software work	Moderate	More: the run, plus two prerequisites
How much JLR data work	Parts with prices we paid	Our full rate book
Does it tell us how accurate we are	Yes, that is the point of it	It relies on that answer
Can we stop part way	Yes, nothing in CAPEE has changed	Yes, it runs alongside
Is it waiting on an unknown	Yes, how CAPEE is built	Yes, the accuracy figure
Is it audit-ready today	Yes, CAPEE keeps the record	No, until rate history and saved costings exist

Read the last two rows together

Option 1 produces the accuracy figure that Option 2 leans on, and Option 1 is audit-ready today because CAPEE keeps the record. Option 2 is not, until the rate history and the saved costings exist. That is the whole argument for this order.

No durations shown. Nothing in either option has been sized by JLR.

RISKS

Risks and open items

Item	Applies to	Detail and what sorts it	Rating
We do not know how accurate it is	Both	Never compared against a price JLR has paid. The Option 1 trial is what settles it.	High
Old costings cannot be reproduced	Option 2	No rate history and no saved costings. Has to be built before bulk is used for anything audited.	High
We do not know how CAPEE is built	Option 1	Holds up the choice of connection route. One answer from IT sorts it.	High
The automated route has gaps	Option 2	No region, no annual volume, no gear. Small work, but a basket is quietly wrong without it.	Medium
The built-in numbers are our estimates	Both	Starting points, not JLR plant measurements. They get replaced as real data comes in.	Medium
Drawing text goes to an AI service	Both	Short extracts only, and it can go through a JLR service. The CAD file itself stays on the server.	Medium
Server type	Option 1	The measuring engine needs a normal Linux server. Easy to confirm now, awkward if we miss it.	Low
The tool is still changing	Both	Six rounds of work in the last few weeks. For any trial we pin a version and stay on it.	Low

One decision deals with the top item

Approving the Option 1 trial turns "we do not know how accurate it is" into a number. Everything else on this list is smaller and has a known fix.

NEXT STEPS

What we need to get going

- **30 to 50 parts with the price we actually paid**
Part, description, annual volume, where it was made, and the price paid. Existing purchase records will do. This is what turns "we think it is about right" into a number, and both options rest on it.
- **Our own rate data**
Material prices, machine rates per hour, labour by grade, electricity and gas. Worth starting now whatever is decided, because both options need it and it is ours to gather.
- **How CAPEE is built**
What it is written in, whether it can call another service on the network, and what kind of server it runs on. Three questions to IT. Unblocks Option 1.
- **A decision on the drawing-reading step**
Either switch off the AI reading and keep the measuring, or run it through a JLR-approved service. The data-flow document is written and covers both.
- **A team and one part type for the trial**
One cost engineer and one commodity family for the duration. Machined parts or pressings would be my choice.

The first two do not wait on any decision made today

Purchase records and our own rates are useful whichever way this goes.

APPENDIX

Where the numbers come from

Every count and timing in this pack was read off the software or measured this month, and checked again when these slides were built.

Statement	How it was checked
19 part types cost through the engine	Traced each one from its screen through to the costing
182 values worked out, 139 land in a named box	Counted in the rules packs for the twelve part types that have them
12 to 69 boxes typed by hand today	Counted the input fields on the machining, casting, moulding and pressing forms
The costing gives the same answer every run	Ran one part five times, identical to the last decimal
Measuring makes no internet connection	Checked every line for outbound calls; there are none
CAD files stay on the server	Measuring happens locally; only drawing text goes to an AI service
1.2 seconds to measure a part	Average over eight real parts, 0.6 MB to 20 MB, this month
500 parts in about five minutes	That average, on two measuring workers, measured not estimated
Six of eight sample parts costed straight through	Two stopped and asked. Our own parts, not JLR parts
Line counts on slide 7	Counted directly in the source files
Settings values on slide 9	Read off the software; each is a single defined value
328 materials, 178 machines, 42 labour rates, 20 regions	Counted in the rate library; rates effective June and July 2026
2,079 automatic tests across 156 files	Full test run at build time, all passing
The engine runs on its own, outside the application	Called it standalone; produced the full eight-bucket answer unchanged

One number we deliberately do not show

The tool has an accuracy report, but no real purchase prices are loaded into it yet. That is ask number one.